A WORKING CHECKLIST

Five Checks That Lie

Five passing checks that proved nothing, found in one person's own pipeline in 48 hours. What each one looks like, how to check for it in yours, and what a real pass looks like instead of a green one.

Every one of these produced a real green signal. Each measured something adjacent to what was actually being cared about. A passing test proves the code behaves the way the test invokes it, not that the program runs.

Esteban Valencia

linkedin.com/in/estebanvalencia

The billed-API mirage

WHAT IT LOOKS LIKE

A test suite passes because it is making real, billed live API calls. It only fails in CI, where there is no key. Locally, everything looks green because the test is quietly talking to a live service instead of testing your code's logic.

HOW TO CHECK FOR IT

> Run the suite with the API key unset, or with network access revoked. If it still passes, it was never testing what you think. If it fails only in that condition, you have found the mirage.

WHAT A REAL PASS LOOKS LIKE

The suite passes against a mock or fixture standing in for the API, with no key and no network required. A separate, clearly labeled integration test covers the live call, and it fails loudly, not silently, when the key is missing.

The URL that resolves to nothing

WHAT IT LOOKS LIKE

A trace URL returns success and resolves to nothing. The check confirmed a URL came back, not that it pointed anywhere real.

HOW TO CHECK FOR IT

> Take the URL your check produced and actually open or fetch it. Confirm it resolves to real content, not a blank page, a 404, or an empty payload dressed up as a success response.

WHAT A REAL PASS LOOKS LIKE

The check asserts on the content the URL resolves to, not on the shape of the string. A URL that comes back but points nowhere fails the check, the same way a URL that never came back would.

The import path only the test runner sees

WHAT IT LOOKS LIKE

Eight demo scripts broke with a ModuleNotFoundError while the suite stayed green. The test runner put the repo root on the import path; running the script directly did not. The suite was exercising an import path the real entry points never use.

HOW TO CHECK FOR IT

> Run every entry point exactly the way a real user or a real deployment would run it, outside your test runner's environment, not through it. If it breaks there and passes in the suite, the suite is testing the runner, not the program.

WHAT A REAL PASS LOOKS LIKE

The suite exercises the same import path your real entry points actually use. A script that works when the test framework runs it and fails when a person runs it directly has not passed anything that matters.

The test that cannot fail

WHAT IT LOOKS LIKE

Fixtures written in markdown for an OCR pipeline. Markdown is already machine-readable text, so the test cannot fail, which means it cannot pass either. It never asked the pipeline to do the job it exists to do.

HOW TO CHECK FOR IT

> Ask one plain question of every test: what would actually make this fail? If you cannot describe a realistic failure, the test is not testing anything, it is just running.

WHAT A REAL PASS LOOKS LIKE

Fixtures use the real input format the pipeline was built for, an actual scanned image for an OCR pipeline, not a transcription of what the image says. The test can genuinely fail, which is what makes a pass mean something.

The single-file grep close

WHAT IT LOOKS LIKE

A task closed after grepping one file for the right constant, while a second render path still showed the old value. The fix existed. It just was not the only place the old value lived.

HOW TO CHECK FOR IT

> Before closing, search every path that could touch the value, not only the one you edited, and check what a user or downstream system actually sees, end to end, not just what the source file now says.

WHAT A REAL PASS LOOKS LIKE

The value shows correctly wherever it is produced or rendered, in every path, not just wherever it happens to be stored.

“If a check has never failed, you have not verified that it works.”

Esteban Valencia · linkedin.com/in/estebanvalencia